AI Infrastructure Fast Track

8 Weeks to Hireable — Skip the Theory. Build the Portfolio. Get the Job.

Triton · FlashAttention · Quantization · PyTorch integration · vLLM contribution · Updated June 04, 2026

8	26	5	1
Weeks	Build Days	Portfolio Projects	OSS Contribution

The goal is top 20% of practical builders — not top 1% of GPU researchers.

You don't need to understand every CUDA optimization. You need to be able to say: 'I built LayerNorm in Triton, benchmarked it, it's 1.4x faster than PyTorch, here's why.' That answer — with a GitHub link — is what gets you an AI infra interview. This plan gets you there in 8 weeks full-time (or 3-4 months part-time). Every day has one deliverable. Commit it to GitHub that day.

The 8-Week Plan at a Glance

Week	Focus	What You Have at the End	Status
Week 0	**PyTorch + GPU basics**	CPU vs GPU benchmark, transformer in PyTorch from scratch	✓
Week 1	**Triton fundamentals**	vector_add, vector_mul, reductions, fused ReLU — all on GitHub	build
Week 2	**Core ML kernels**	Softmax (online), LayerNorm, RMSNorm, GELU — all benchmarked	build
Week 3	**GEMM + Attention**	Tiled matmul + FlashAttention forward, benchmark table	build
Week 4	**Quantization**	INT8 + FP8 quantized linear layer, 1.5x throughput vs FP16	build
Week 5	**PyTorch integration**	All kernels wrapped as torch ops, torch.compile compatible	build
Weeks 6-7	**Serious project**	One of the 5 portfolio projects — with benchmark numbers	build
Week 8	**OSS + interview prep**	vLLM codebase read, PR opened, interview answers ready	→

W0 PYTORCH + GPU BASICS

Days 1–3 · GPU tensors, transformer architecture, memory bandwidth · ~8 hrs total

Day	Focus	Deliverable — commit this to GitHub	Time
D1	**PyTorch + GPU tensors**	x = torch.randn(4096, device='cuda'). Benchmark: CPU vs GPU matmul at N=512/1024/4096. Understand why GPU wins at N>512.	**3 hrs**
D2	**Transformer architecture**	Visualize: Embedding → Attention (Q/K/V) → MLP → Output. Implement each layer in PyTorch from scratch. No libraries.	**3 hrs**

Day	Focus	Deliverable — commit this to GitHub	Time
D3	**Memory bandwidth basics**	torch.cuda.memory_allocated(), bandwidth = bytes / time. Measure bandwidth of different ops: elementwise vs matmul vs softmax.	**2 hrs**

W1 TRITON FUNDAMENTALS

Days 4–7 · First kernels, masking, reductions, fused ops · ~10 hrs

Day	Focus	Deliverable — commit this to GitHub	Time
D4	**Triton install + hello**	@triton.jit, tl.program_id, tl.arange, tl.load, tl.store — write vector_add.py. Compare to torch equivalent.	**3 hrs**
D5	**Masking pattern**	mask = offsets < n — prevent out-of-bounds. Safe vector_mul.py. Run on non-power-of-2 sizes. Verify against torch.	**2 hrs**
D6	**Reductions**	tl.sum, tl.max — row-wise and column-wise. row_sum.py: each program handles one row. Verify with torch.sum.	**2 hrs**
D7	**Fused elementwise**	Fuse ReLU + scale + bias into one kernel. Benchmark fused vs 3 separate torch ops. Measure HBM bandwidth saved.	**3 hrs**

W2 CORE ML KERNELS

Days 8–12 · Softmax (online algo), LayerNorm, RMSNorm, GELU · ~13 hrs

Day	Focus	Deliverable — commit this to GitHub	Time
D8	**Numerically stable softmax**	Subtract row max before exp. Two-pass: max scan then exp/sum. softmax.py. Benchmark vs torch.softmax at seq_len=512/2048.	**3 hrs**
D9	**Online softmax**	Single-pass: track (running_max, running_sum) simultaneously. This IS the FlashAttention insight — understand it now.	**3 hrs**
D10	**LayerNorm**	Mean + variance in one pass (Welford). Normalize, scale, bias. layer_norm.py. Verify grads with torch.autograd.gradcheck.	**3 hrs**
D11	**RMSNorm**	No mean subtraction — just RMS. Used in LLaMA/Mistral/Gemma. Faster than LayerNorm. rms_norm.py. Benchmark both.	**2 hrs**
D12	**GELU + SiLU fused**	Fuse GELU/SiLU + dropout in one kernel. These are the MLP activation functions in every transformer. Benchmark.	**2 hrs**

Day 9 (online softmax) is the most important session in weeks 1-2.

Online softmax — tracking (running_max, running_sum) in a single pass without storing the full array — is the algorithmic insight behind FlashAttention. If you understand it here, FlashAttention in Week 3 will click immediately. If you skip it, you'll copy code without understanding it.

W3 GEMM + FLASHATTENTION

Days 13–17 · Tiled matmul, autotuning, naive attention, FA forward + causal mask · ~14 hrs

Day	Focus	Deliverable — commit this to GitHub	Time
D13	**Tiled GEMM**	Load A-tile [M,K] + B-tile [K,N] into SRAM, tl.dot(), accumulate fp32, store fp16. matmul.py. Compare vs torch.matmul.	**3 hrs**
D14	**@triton.autotune**	Search BLOCK_M, BLOCK_N, BLOCK_K, num_stages, num_warps automatically. Add to matmul.py. This is how you match cuBLAS.	**2 hrs**
D15	**Naive attention**	QK^T / sqrt(d) → softmax → V. Naive: materialize full NxN matrix. attention_naive.py. Works, slow, uses O(N²) memory.	**2 hrs**
D16	**FlashAttention forward**	Tile Q/K/V, run online softmax across K-blocks — never materialize NxN. flash_attn_fwd.py. Verify vs F.scaled_dot_product_attention.	**4 hrs**
D17	**Causal masking + benchmark**	Add causal mask (upper-triangular -inf). Benchmark: naive vs FA at seq_len=512/1024/2048/4096. Measure memory + throughput.	**3 hrs**

Day 16 (FlashAttention forward) is the hardest day in the whole course.

Budget a full day. Read the FlashAttention paper first (even just the abstract + Figure 1). The tiling pattern: for each Q-block, loop over all K/V-blocks, update online softmax state, accumulate weighted V. If you get it working and verified against F.scaled_dot_product_attention, you're hireable at inference companies.

W4

QUANTIZATION

Days 18–21 · INT8 kernels, FP8 basics (H100+), quantized linear layer · ~11 hrs

Day	Focus	Deliverable — commit this to GitHub	Time
D18	**INT8 quantization kernel**	Absmax quantize: scale = max(|x|) / 127. quantize.py. Dequantize kernel. Verify round-trip error < 0.01.	**3 hrs**
D19	**INT8 matmul**	Quantize inputs → INT8 matmul → dequantize output. int8_matmul.py. Benchmark vs FP16 matmul — show 1.5-2x throughput.	**3 hrs**
D20	**FP8 basics**	E4M3 format (H100+). torch.float8_e4m3fn. Scale factor per tensor. fp8_linear.py. Benchmark vs BF16 on H100.	**2 hrs**
D21	**Quantized linear layer**	Full quantized_linear.py: INT8 weights + activations, scale factors, bias in FP16. This is what bitsandbytes does inside.	**3 hrs**

W5

PYTORCH INTEGRATION + TESTING

Days 22–26 · Custom ops, autocast, torch.compile, benchmarking suite · ~11 hrs

Day	Focus	Deliverable — commit this to GitHub	Time
D22	**PyTorch custom op**	torch.autograd.Function: @staticmethod forward() + backward() calling your Triton kernels. attention_op.py.	**3 hrs**
D23	**Autocast dispatch**	Handle bf16/fp16/fp32: check input.dtype, dispatch correct kernel. torch.cuda.amp.autocast compatibility.	**2 hrs**
D24	**torch.compile integration**	@torch.compile calls Triton kernels directly via Torch Inductor. Test your ops compile cleanly. Fix any graph breaks.	**2 hrs**

Day	Focus	Deliverable — commit this to GitHub	Time
D25	**triton.testing.do_bench**	Proper benchmarking: warmup=25, rep=100, percentiles. Build bench/run_all.py that tests every kernel you've written.	**2 hrs**
D26	**Debugging toolkit**	TRITON_INTERPRET=1 for CPU debug. torch.allclose() correctness checks. Build a test_suite.py for all your kernels.	**2 hrs**

W6-7 SERIOUS PROJECT

2 weeks · Pick ONE of the 5 projects below · No distractions · Benchmark numbers + clean README

Day	Focus	Deliverable — commit this to GitHub	Time
W6-7	**Serious project**	Pick ONE from the 5 portfolio projects below. Two weeks, no distractions. Benchmark numbers + clean README + GitHub.	**2 wks**

W8 OPEN SOURCE + INTERVIEW PREP

Days 1–5 · Read vLLM, find a PR, interview answers, apply

Day	Focus	Deliverable — commit this to GitHub	Time
D1	**Read vLLM codebase**	vllm/attention/ops/ — read the Triton paged attention kernel. Understand PagedAttention: block table, slot mapping.	**1 day**
D2	**Read SGLang or TGI**	SGLang's attention kernels or TGI's Triton ops. Find one thing you'd improve or a comment that's wrong.	**1 day**
D3	**Find a good first issue**	github.com/vllm-project/vllm/issues?label='good+first+issue' — find something involving Triton kernels or benchmarks.	**1 day**
D4	**Interview prep**	Practice explaining: 'Why is your softmax faster?' 'What's the memory bottleneck in attention?' 'How does FlashAttention work?'	**1 day**
D5	**Apply + reach out**	Apply to: CoreWeave, Baseten, vLLM team (Anyscale), Together AI, Modal, Replicate, any AI lab infra team.	**ongoing**

The 5 Portfolio Projects — Pick One for Weeks 6-7

Pick the one that aligns with the role you're targeting. Project 5 (vLLM contribution) is the most impressive but hardest. Project 2 (FlashAttention) is the most universally recognized. Do only ONE — two half-finished projects are worse than one polished one.

PROJECT 1 — Triton Kernel Library

GELU · Softmax · LayerNorm · RMSNorm · RoPE · all benchmarked vs PyTorch

What you build:

1. One file per kernel: gelu.py, softmax.py, layer_norm.py, rms_norm.py, rope.py
2. Each with @triton.autotune + correctness test + triton.testing.do_bench benchmark
3. README table: kernel vs torch baseline, speedup, memory bandwidth utilization
4. CI that runs tests on every push (GitHub Actions + GPU runner or local)

Key files:

- kernels/ — one .py per kernel
- bench/run_bench.py
- tests/test_all.py
- README.md — benchmark table

Say this in interviews:

✦ *Built Triton kernel library: 5 production kernels (GELU, Softmax, LayerNorm, RMSNorm, RoPE), each autotuned and benchmarked vs PyTorch baseline — 1.2-1.8x speedup on A100*

PROJECT 2 — FlashAttention Implementation

Tiled · Online softmax · Causal mask · Memory benchmark vs naive

What you build:

1. flash_attn_fwd.py: tiled attention, online softmax, causal + non-causal
2. Verify vs F.scaled_dot_product_attention to 1e-3 tolerance
3. Benchmark table: naive attention vs FA at seq_len 512/1024/2048/4096 — memory + throughput
4. PyTorch wrapper: torch.autograd.Function + autocast dispatch

Key files:

- flash_attn/fwd.py
- flash_attn/interface.py
- bench/memory_bench.py
- README.md

Say this in interviews:

✦ *Implemented FlashAttention in Triton: O(N) memory vs O(N²) naive; 2.1x throughput at seq_len=4096, 3.2x memory reduction on A100 BF16*

PROJECT 3 — Quantized Linear Layer

INT8 + FP8 weights · scale factors · 1.5-2x throughput vs FP16

What you build:

1. INT8 quantization kernel: absmax per-tensor and per-channel scale
2. INT8 matmul kernel: quantize → compute → dequantize in Triton
3. FP8 variant for H100 (E4M3 format)
4. Drop-in replacement for torch.nn.Linear with benchmark comparison

Key files:

- quant/quantize.py
- quant/int8_linear.py
- quant/fp8_linear.py
- bench/quant_bench.py

Say this in interviews:

> *Built quantized linear layer in Triton: INT8 + FP8, per-channel scales, 1.7x throughput vs FP16 on A100; error < 0.5% vs FP32 baseline*

PROJECT 4 — Tiny vLLM-style Inference Engine

KV cache · Paged attention · Sampling · Triton throughout

What you build:

1. KV cache: pre-allocated blocks, write-at-position Triton kernel
2. Paged attention: block table lookup + gather kernel for decode step
3. Greedy + top-p sampling kernels in Triton
4. End-to-end: tokenize → prefill → decode loop → detokenize — run a real model

Key files:

- engine/kv_cache.py
- engine/paged_attn.py
- engine/sampling.py
- engine/runner.py

Say this in interviews:

> *Built mini LLM inference engine: Triton KV cache + paged attention + sampling; ran Qwen2.5-1.5B end-to-end, matched HuggingFace output, 1.4x lower decode latency*

PROJECT 5 | **Contribute to vLLM** | Real open-source contribution · Most impressive on resume

What you build:

1. Read vllm/attention/ops/triton_flash_attn.py end to end — understand every line
2. Find: benchmark that's missing, kernel config that's suboptimal, bug in edge case
3. Open PR: fix + test + benchmark numbers showing improvement
4. Even a small merged PR to vLLM is worth more than 3 personal projects

Key files:

- Your fork of vllm
- PR with benchmark numbers
- Link in your resume/LinkedIn

Say this in interviews:

> *Contributed [specific fix] to vllm-project/vllm — merged PR improving [kernel / benchmark / edge case], reviewed by vLLM core team*

What Interviewers Actually Ask — and How to Answer

They don't quiz you on GPU architecture. They ask if you can explain what you built.

Every answer should follow: what problem, what technique, what the result was (with numbers). 'I implemented online softmax in Triton, fused it with the attention score computation, and measured 2.1x throughput vs PyTorch SDPA at seq_len=4096 on A100.' That answer gets callbacks.

Question	What a Good Answer Covers
Have you optimized a kernel before?	Yes — say which kernel, what technique (tiling, fusion, online algorithm), what the speedup was.
Walk me through your FlashAttention implementation.	Explain: tiling strategy, why you never materialize NxN, online softmax state (max + sum per row), causal mask.
What's the memory bottleneck in standard attention?	$O(N^2)$ attention matrix — grows quadratically with sequence length, kills throughput at N>1024.
Why is your softmax faster than PyTorch's?	Fused kernel: one read + one write instead of separate max, exp, sum, divide passes.
What's the difference between INT8 and FP8?	INT8: fixed-point, 8 bits, for weights+activations. FP8 (E4M3/E5M2): floating-point, H100+ hardware support, 2x throughput vs BF16 for matmuls.

What They Do NOT Ask (don't waste time on these):

~~Explain warp divergence mathematically~~	~~What are the exact CUDA occupancy formulas?~~	~~Describe the L2 cache associativity on Hopper~~	~~What is the difference between cudaLaunchKernel and <<<>>> syntax?~~	~~Explain PTX assembly~~

5 Papers to Read (in this order)

Don't read them all before building — read each one when you reach the relevant week.

Paper	When to Read + Why
FlashAttention (Dao et al. 2022)	THE paper. Read it before building Session 16. Understand the tiling argument.
FlashAttention-2 (Dao, 2023)	Explains the backward pass and parallelization improvements. Read before building backward.
PagedAttention (Kwon et al. 2023)	The vLLM paper. Explains block table, slot mapping, KV cache fragmentation problem.
DeepSeek-V3 Technical Report (2024)	Modern MoE + MLA attention. Shows what production AI infra actually looks like.
AWQ (Lin et al. 2023)	Per-channel INT4 weight quantization. Explains why per-channel > per-tensor for LLMs.

After this: read vLLM source, then SGLang, then TensorRT-LLM. CUDA depth comes naturally after 3-6 months working with these systems — you'll know exactly what to learn because production will tell you.